



8a-vm-hw

Process'leri RAM'e Sığdırma

Bilgisayarın ana belleği (ram), devasa ve bomboş bir alandır. İşletim sisteminin görevi ise aynı anda çalışan onlarca process'e ram üzerinde yer tahsis etmektir. Fakat bunu yaparken çözülmesi gereken devasa krizler vardır.

Sanal Hafızanın Oluşturduğu Problemler

Process'lerin güvenliğinin ve korunmasının sağlanması için ram'in sanallaştırılmasına ihtiyaç duyulur.

Adres Bağlama Sorunu

Bir program yazıp derlediğinde (Compile Time), hafıza adresleri sabittir. Kodda "100. adrese git" yazıyorsa ve sen bu programı RAM'de 5000. adrese koyarsan program patlar.

Yazılımsal Çözüm

Buna ilk çözüm olarak **Yükleme Zamanı (Load Time)** bağlaması bulunmuştur: Program RAM'e yüklenirken, işletim sistemi kodun içindeki tüm "100" yazan yerleri tek tek bulup "5100" olarak günceller (yama yapar).

Neden Yetersiz? Ya program çalışırken RAM'de başka bir yere taşımamız gerekirse? Veya RAM'de bitişik halde o kadar büyük bir boş yer yoksa? Bu yöntem orada tıkanır.

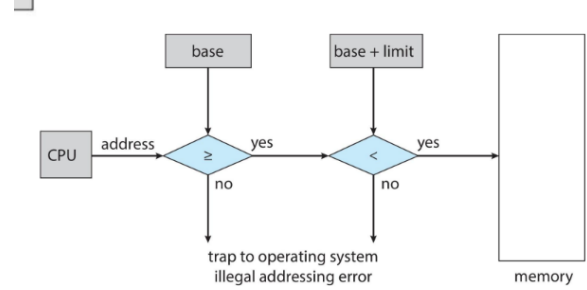
Donanımsal Çözüm

İşlemciye iki yeni güvenlik görevlisi (kayıtçı/register) eklenir.

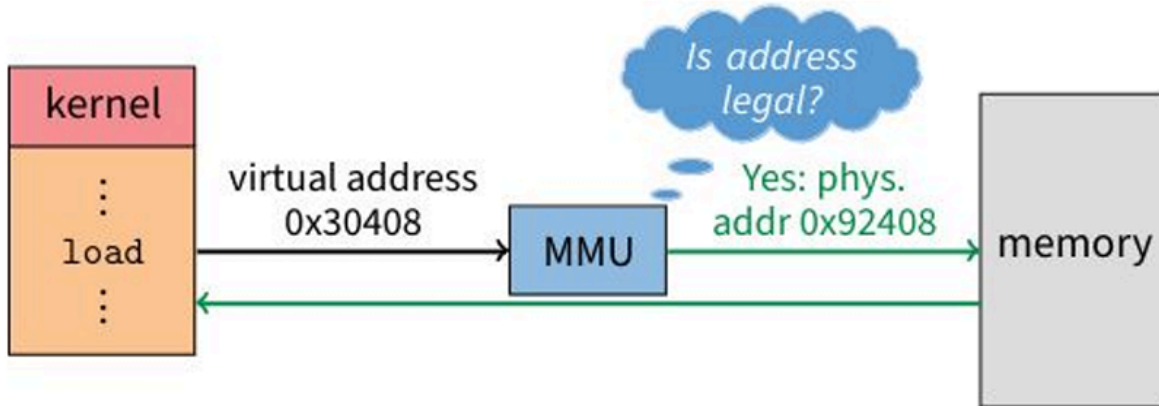
- **Base (Taban):** Programın aradaki başlangıç noktası.

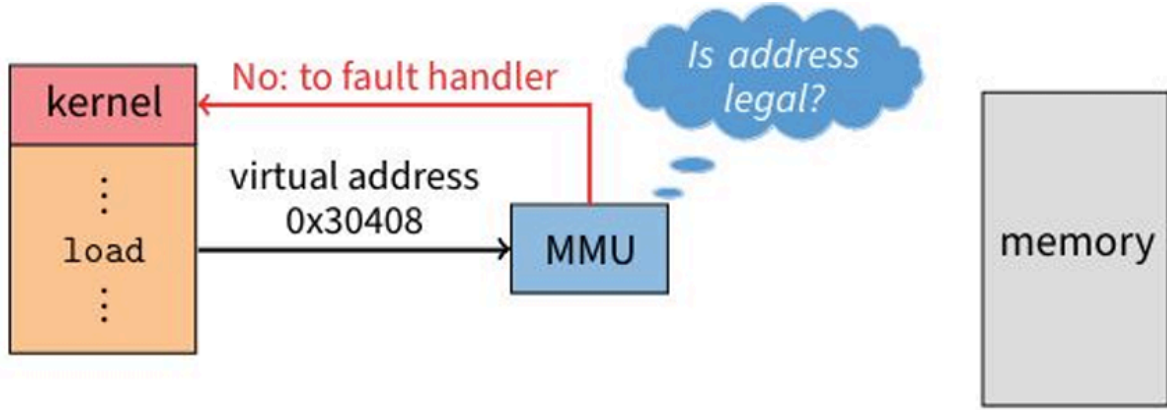
- **Bound / Limit (Sınır):** Programın arazisinin büyüklüğü. İşlemci (CPU) kod çalıştırırken "Hafızadaki 346. adrese git" dediğinde, bu iki güvenlik görevlisi araya girer:

1. Önce "346" sayısı senin Base (örneğin 14000) değerinle toplanır. Gerçek adres 14346 olur.
2. Sonra bu adresin senin Sınır (Limit) değerini aşıp aşmadığı kontrol edilir.
3. Eğer aşıyorsa donanım derhal "Dur!" diyerek İşletim Sistemine "İllegal Adres Hatası (Trap)" fırlatır.



- **MMU (Memory Management Unit):** Base+Bound toplamasını ve kontrolünü milisaniyeler içinde yapan donanım parçasına **MMU (Memory Management Unit)** diyoruz.
 - **Mantıksal (Sanal) Adres:** CPU'nun (ve senin programının) gördüğü sahte adrestir. Senin programın kendini dünyadaki tek program sanır ve hep 0. adresten başladığını düşünür.
 - **Fiziksel Adres:** Gerçek RAM'deki adrestir. Sadece MMU ve RAM bu adresi bilir, programın ruhu bile duymaz.





Segmentasyon ve Sayfalama

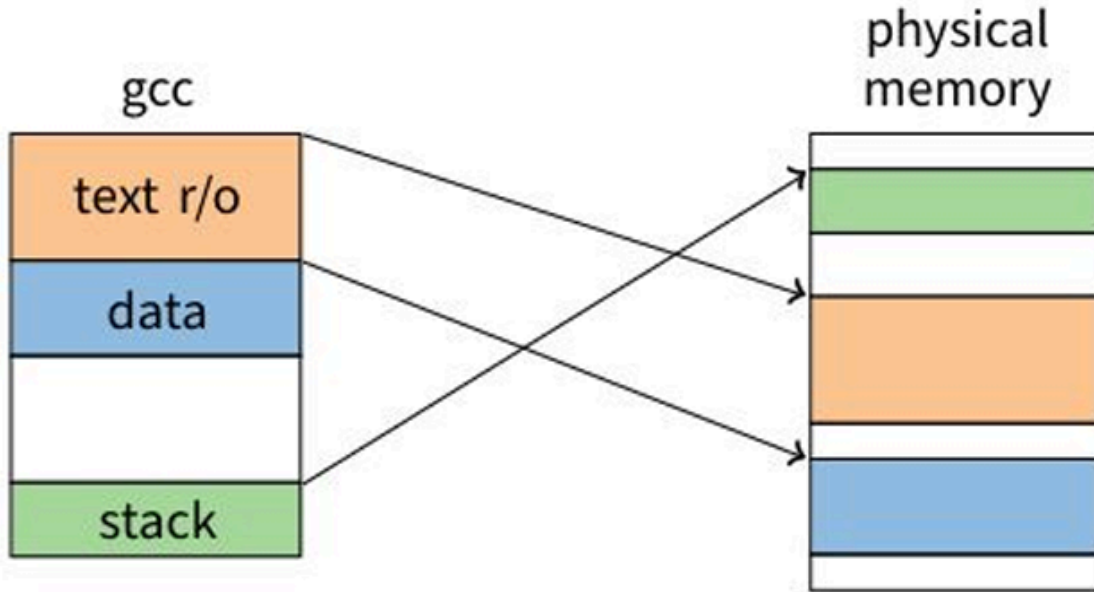
Eski sistemde programa "Al sana kocaman, tek parça bir arazi, burada yaşa" diyorduk. Ama programın içine baktığımızda aslında üç farklı mahalleden oluştuğunu görürüz:

1. **Kod (Text):** Programın çalıştırılabilir komutları.
2. **Veri (Data):** Değişkenlerin tutulduğu yer.
3. **Yığıt (Stack):** Fonksiyonların ve geçici verilen çalıştığı, sürekli büyüyüp küçülenn yer.

İşte buna **Segmentasyon** diyoruz. Artık her mahallenin (segmentin) kendi Taban (Base) ve Sınır (Bound) değeri var.

Böylece Kod segmentini RAM'in bir köşesine, Stack segmentini bambaşka bir köşesine koyabiliriz.

Hatta iki farklı Google Chrome sekmesi açıldığında, ikisi de aynı "Kod" segmentini paylaşabilir, RAM'den devasa tasarruf ederiz!



Dış Parçalanma (External Fragmentation)

Segmentasyon kulağa harika geliyor ama zamanla RAM'i bir İsviçre peynirine çevirir. Neden mi? Çünkü her programın segment boyutu birbirinden farklıdır. Programlar açılıp kapandıkça RAM'de irili ufaklı yüzlerce "boşluk" oluşur.

İşletim sistemi bu irili ufaklı boşluklara yeni bir segment yerleştireceği zaman üç mantık güder:

- **First-fit (İlk Uyan):** Baştan tarar, sığdığı ilk deliğe girer (Hızlıdır).
- **Best-fit (En İyi Uyan):** Tam tamına sığacağı en küçük deliği arar. (Kötüdür, çünkü geriye hiçbir işe yaramayacak kadar küçük, mikroskobik delikler bırakır).
- **Worst-fit (En Kötü Uyan):** Gidip bilerek en büyük deliğe yerleşir ki, geriye kalan boşluk ileride başka bir işe yarayabilsin.

Kesin ve Kusursuz Çözüm: Sayfalama (Paging)

- Fiziksel RAM'i eşit boyutlu (örneğin 4 KB) kutulara bölerler. Bunlara **Çerçeve (Frame)** denir.

- Senin mantıksal programını da aynı eşit boyuttaki (4 KB) parçalara bölerler. Bunlara da **Sayfa (Page)** denir.

Artık programının 1. Sayfası RAM'deki 5. Çerçeveye, 2. Sayfası RAM'deki 900. Çerçeveye gidebilir. Hiçbir sayfanın yan yana durmasına gerek yoktur! Bütün yapboz parçaları aynı boyutta olduğu için Tetris oynamaya gerek kalmaz. RAM'de boş bir çerçeve varsa, herhangi bir sayfa oraya cuk diye oturur. **Dış Parçalanma (External Fragmentation) tamamen yok edilmiştir!**

Sayfalamanın Bedeli: İç Parçalanma (Internal Fragmentation)

Sayfalamanın boyutu sabittir (4 KB). Diyelim ki senin programının küçücük bir kod parçası var ve sadece 1 KB yer kaplıyor. İşletim sistemi ona mecburen tam bir Sayfa (4 KB) vermek zorundadır. Senin 1 KB'lık kodun o sayfaya girer, geriye kalan 3 KB ise o sayfanın içinde hapsolür, başka hiçbir program o sayfayı kullanamaz.

Kullanılmayan bu alan kutunun *içinde* kaldığı için buna **İç Parçalanma (Internal Fragmentation)** diyoruz.

Page'lerin Takibi

İşletim sistemlerinin programları "Sayfalara", fiziksel RAM'i ise "Çerçevelere" (Kutulara) böldüğünü ve böylece boşlukları (Dış Parçalanmayı) yok ettiğimizi bir önceki aşamada konuşmuştuk. Peki ama bu kadar çok parçayı kim, nasıl takip edecek?

Sayfa Tablosu

Programın parçalandı ve RAM'in her yerine rastgele dağıldı. İşletim sistemi, her program için özel bir **Sayfa Tablosu** tutar.

İşlemci (CPU) kod çalıştırırken "Bana programın 3. sayfasını getir" dediğinde, hemen bu deftere bakılır: *"Ah, 3. sayfa RAM'deki 15. çerçevenin içindeymiş"* Böylece Sanal Adres (Sanal Sayfa Numarası - VPN), Fiziksel Adrese (Fiziksel Çerçeve Numarası - PPN) çevrilir.

Hız Felaketi ve TLB (Donanımsal VIP Not Defteri)

Ancak ortada korkunç bir sorun vardır: Bu "Adres Defteri" de fiziksel RAM'in içinde durmaktadır! İşlemci bir değişkene ulaşmak istediğinde önce RAM'e gidip deftere (Sayfa Tablosuna) bakmalı, adresi öğrenmeli ve sonra *tekrar* RAM'e gidip o adresten değişkeni almalıdır. Yani 1 iş için RAM'e 2 kez gidilir. Bu, bilgisayarı yarı yarıya yavaşlatır!

Bunu çözmek için işlemcinin kalbine **TLB (Translation Lookaside Buffer)** adında inanılmaz hızlı, ama kapasitesi çok küçük özel bir donanım eklenir. TLB, sık kullanılan adres çevirilerini hafızasında tutan bir "VIP Not Defteridir".

Etkin Erişim Süresi (EAT) Formülü:

- Diyelim ki belleğe bir kez gitmek 10 nanosaniye (ns) sürüyor.
- Aradığımız adresi doğrudan TLB'de bulursak (Hit), hemen adresi alır 10ns'de RAM'deki veriye ulaşırız.
- TLB'de bulamazsak (Miss), mecbur deftere bakmak için 10 ns, veriyi almak için 10 ns, toplam 20 ns harcarız.
- Eğer TLB'de bulma oranımız %80 ise ortalama hızımız:
 $(0.80 \times 10) + (0.20 \times 20) = 12 \text{ ns}$ olur. Yani o minicik donanım, RAM'e iki kere gitme çilesini ortadan kaldırıp sistemi kurtarır.

Boyut Kabusu ve Çok Katmanlı (Hiyerarşik) Tablolar

Diyelim ki C dilinde basit bir program yazdın ve çalıştırdın. İşletim sistemi her programa (sanki dünyadaki tek program oymuş gibi) devasa bir 4 GB'lık sanal hafıza alanı (arazi) sunar.

Eğer biz programın haritasını tutmak için **tek seviyeli düz bir Sayfa Tablosu (Adres Defteri)** kullanırsak, işletim sistemi şunu yapmak zorundadır: *"Bu programın 4 GB'lık arazisindeki HER BİR KARIŞ için deftere bir satır açmalıyım."* Bunu yaptığında, programın adres defteri tam **4 Megabayt (MB)** yer kaplar.

- Ama dur bir saniye! Senin yazdığın program belki de sadece 1 MB'lık minicik bir yer kullanıyor. Geriye kalan 3.99 GB'lık alan tamamen boş!
- Buna rağmen o düz defterde, boş alanlar için bile milyonlarca boş satır tutmak zorundasındır. Sadece 100 tane sekme veya program açsan, sırf "içi boş adres

defterleri" yüzünden RAM'inden 400 MB çöpe gider.

Mühendisler bu israfı önlemek için haritayı tek bir defter yapmak yerine **İki Seviyeli (Hiyerarşik)** bir yapıya bölerler.

- 1. Seviye - Ana Dizin (Outer Page Table): Bu çok küçük bir rehberdir (örneğin sadece 4 KB boyutunda). İçinde sadece 1024 satır vardır ve her satır 4 MB'lık koca bir bölgeyi temsil eder.
- 2. Seviye - Alt Tablolar (Inner Page Tables): Ana dizindeki her bir satır, kendi bölgesinin detaylı sokak haritası olan bir "Alt Tabloya" işaret eder.

İşte hiyerarşinin asıl parlama noktası (sınavda puan getiren mantık) burasıdır: Senin o minicik programın 4 GB'lık arazinin sadece başındaki 1 MB'ı kullanıyordu ya;

- İşletim sistemi Ana Dizin'e bakar. Sadece **1. satırın** dolu olduğunu (senin kodlarının orada olduğunu) görür.
- Sadece o 1. satır için gidip bir **Alt Tablo (2. Seviye defter)** yaratır.
- Ana Dizin'deki geriye kalan 1023 satırın karşısına **"BOŞ" (Null)** yazar.
- **ALTIN KURAL:** İşletim sistemi, karşısında "BOŞ" yazan yerler için gidip de fiziksel RAM'de bir alt tablo oluşturmaz! O içi boş 1023 cildi **hiçbir zaman matbaada basmaz.**

64-Bit Sistemlerde Neden 4 Seviye Var?

Kullandığın 64-bit işlemcilerdeki adres alanı o kadar devasadır ki (16 Exabyte), bunu ikiye bölmek bile yetmez. Bu yüzden dizinin dizininin dizini şeklinde **4 Seviyeli (Level-4)** bir haritalama yaparlar. Mantık hep aynıdır: Kullanılmayan koca boşlukların haritası fiziksel RAM'de asla yaratılmaz.

Tersine Sayfa Tablosu (Inverted Page Table)

Her program için ayrı bir adres defteri tutmak yerine, mühendisler *"Neden RAM'in merkezine tek bir büyük beyaz tahta koymuyoruz?"* dediler.

Bu beyaz tahtada programlar değil, doğrudan RAM'deki çerçeveler listelenir. *"1. Çerçevede A programının 5. sayfası var. 2. Çerçevede B programının 1. sayfası var."*

Bu yapıya **Tersine Sayfa Tablosu** denir. Program sayısına değil, bilgisayarındaki fiziksel RAM'in boyutuna bağlı olduğu için devasa bir hafıza tasarrufu sağlar. (Oracle SPARC Solaris gibi sistemler bunu kullanır).

Geçerlilik Tiki (Valid/Invalid)

Sayfa tablosundaki her adresin yanında bir "Geçerli/Geçersiz" (Valid/Invalid) tiki vardır. Eğer tik geçersizse, bu *"Sayfa şu an RAM'de yok, ya yasa dışı bir yer ya da hard diske atılmış"* demektir.

Paylaşılan Kütüphaneler

Diyelim ki arka planda iki tane Word dosyası açık. İkisi de aynı altyapı kodlarını (örneğin libc kütüphanesini) kullanıyor. İşletim sistemi bu kodu RAM'e iki kere yüklemek yerine 1 kere yükler. Her iki programın da Adres Defterinde, o kütüphanenin adresi **aynı fiziksel çerçeveyi** gösterir!

Sabit Diskin Ram Gibi Kullanılması (Swapping)

RAM dolarsa işletim sistemi sabit diski sanki yedek bir RAM'miş gibi kullanır.

Standart Swapping

RAM'i inanılmaz hızlı çalışan ama çok küçük bir çalışma masası, SSD'yi ise devasa ama yavaş bir depo olarak düşün. Masada hiç yer kalmadığında işletim sistemi şunu yapar:

- O an hiç işlem yapmayan, arka planda uyuyan koca bir programı masadan alır ve tamamen depoya (Disk'e) kaldırır. Buna **Swap Out (Dışarı Takas)** diyoruz.
- Masada açılan o devasa boşluğa da çalışmak isteyen yeni programı koyar.
- Eski programa tekrar tıkladığında, onu depodan geri getirir. Buna da **Swap In (İçeri Takas)** denir.

Standart Swapping'in Dezavantajları

Kağıt üzerinde harika dursa da, koca bir programı tek parça halinde diske yazıp okumak bir hız felaketidir.

- **Zaman Kaybı (Context Switch Felaketi):** Sadece 100 MB'lık bir programı yavaş bir diske yazmak 2 saniye, oradan başka bir 100 MB'ı RAM'e almak 2 saniye sürse, bilgisayar 4 saniye boyunca kilitlenir (donar). Normalde donanım üzerindeki bu geçişlerin milisaniyeler sürmesi gerekir!
- **I/O Tehlikesi:** Ya o program diske taşındığı sırada internetten bir dosya indiriyorsa veya klavyeden veri bekliyorsa? Program depoya kalktığında, ağdan gelen veriler RAM'deki o eski boşluğa (belki de bambaşka bir programın üzerine) yazılır ve sistem tamamen çöker.

Sayfalarla Swapping (Page Out, Page In)

- Bir programı tamamen diske atmak yerine, sadece o programın "uzun süredir hiç dokunulmayan" 4 KB'lık minik **sayfalarını (pages)** diske atarlar (Page Out).
- Program o sayfaya ihtiyaç duyduğunda da sadece o 4 KB'lık minik parçayı RAM'e geri çekerler (Page In)

Mobil Cihazlarda Neden Swapping Yoktur?

- Telefonlar depolama için Flash Hafıza kullanır ve flash hafızaların "belirli bir yazma/silme ömrü" vardır.
- Eğer işletim sistemi sürekli RAM'den diske sayfa atıp durursa, telefonun hafıza çipi çok çabuk yıpranır ve bozulur.
- Bunun yerine iOS ve Android çok daha gaddardır: RAM dolduğunda arka plandaki uygulamayı diske atmakla uğraşmaz, uygulamaya "Bana yer aç" der, açmazsa da uygulamayı doğrudan öldürür (Terminate).